

Campus Café Ordering System

Final Team Project | 3 Weeks

Project Overview

The Final Project team will be in pairs, and with someone other than the person(s) you teamed up in the previous 3 weeks. You and your partner will build one piece of a terminal-based café ordering system. In Week 1 every team builds and presents their class to the class. In Week 2 each team selects the best version of each class from across all teams and wires them together into a working app. In Week 3 you polish, handle edge cases, and demo the final product.

What the finished app does:

- Displays a menu of food and drinks with prices
- Lets a customer add items to a cart and see a running total
- Tracks stock and prevents ordering items that are out of stock
- Prints an itemised receipt at checkout
- Shows a session summary (total orders, total revenue) when the app exits

Teams & Class Assignments

Teams of 2. Each team chooses ONE class among A, B, and C to build in Week 1. Multiple teams may build the same class — that is intentional. Every team will have one person build the one class (A or B or C) and one person take on the Role D (App Controller) responsibility.

Role	Class	File	Responsibility
A	MenuItem	menu.py	Defines what a menu item is. Stores all menu data.
B	Order	order.py	Manages the customer's cart. Calculates totals.
C	Inventory	inventory.py	Tracks stock levels. Prevents out-of-stock orders.
D	App Controller	main.py	Connects all classes. Runs the main loop. Assigned in Week 2.

The Shared Contract — Read This First

Because teams will swap each other's classes in Week 2, every implementation **MUST** follow this contract exactly. Method names, parameters, and return types are not negotiable. Everything else (internal logic, variable names, extra helper methods) is up to you.

Role A — MenuItem class (menu.py)

Each menu item must be an object. The `MENU` variable at the top of `menu.py` must be a list of at least 6 `MenuItem` objects covering at least 2 categories (e.g. Drinks, Food).

Attribute	Type	Description
<code>self.name</code>	<code>str</code>	The item name, e.g. "Latte"
<code>self.price</code>	<code>float</code>	The price in dollars, e.g. 4.50
<code>self.category</code>	<code>str</code>	The category, e.g. "Drink" or "Food"

Method signature	Returns	Description
<code>__str__(self)</code>	<code>str</code>	Returns a formatted string, e.g. Latte (Drink) - \$4.50
<code>display_menu(menu)</code>	<code>None</code>	Prints all items grouped by category with prices. Class-level or standalone function.
<code>get_item(menu, name)</code>	<code>MenuItem</code> or <code>None</code>	Searches the <code>MENU</code> list and returns the matching item, or <code>None</code> if not found.

Role B — Order class (order.py)

Manages the customer's current cart during a session. The cart is stored as `self.items` — a list of `MenuItem` objects.

Attribute	Type	Description
<code>self.items</code>	<code>list</code>	A list of <code>MenuItem</code> objects currently in the cart. Starts empty.

Method signature	Returns	Description
<code>add_to_order(item)</code>	<code>None</code>	Appends a <code>MenuItem</code> to <code>self.items</code> .
<code>get_total()</code>	<code>float</code>	Returns the sum of all item prices. Returns 0.0 for an empty cart.
<code>display_order()</code>	<code>None</code>	Prints each item and its price, then the running total.
<code>clear_order()</code>	<code>None</code>	Empties <code>self.items</code> . Called after a successful checkout.

Role C — Inventory class (inventory.py)

Tracks how many of each item are in stock. Stock is stored as `self.stock` — a dict mapping item name (string) to quantity (int).

Attribute	Type	Description
<code>self.stock</code>	<code>dict</code>	Keys are item names (str), values are quantities (int). e.g. {"Latte": 10}

Method signature	Returns	Description
<code>check_stock(item)</code>	<code>bool</code>	Returns True if item.name has stock > 0, False otherwise.
<code>deduct_stock(item)</code>	<code>None</code>	Reduces stock for item.name by 1. Only call after <code>check_stock()</code> returns True.
<code>restock(item_name, amount)</code>	<code>None</code>	Increases stock for item_name by amount.
<code>stock_report()</code>	<code>None</code>	Prints all items and their current stock level.

Week 1 — Build & Showcase Your Class

Your goal this week:

- Build your team's assigned class following the shared contract exactly
- Write a short test block at the bottom of your file that proves each method works
- Prepare a 3-minute walkthrough to present to the class at the end of the week

Step-by-step tasks:

Role A — MenuItem

1. Create `menu.py`. Define the `MenuItem` class with `name`, `price`, and `category` attributes in `__init__`.
2. Write `__str__` so that `print(item)` gives a clean, readable line.
3. Create a `MENU` list with at least 6 items across 2+ categories. Hard-code the data directly in the file.
4. Write `display_menu()` — prints items grouped by category.
5. Write `get_item(menu, name)` — returns the `MenuItem` whose name matches, or `None`.
6. Test block: call `display_menu(MENU)`, then call `get_item(MENU, "Latte")` and print the result.

Role B — Order

1. Create `order.py`. Define the `Order` class with `self.items = []` in `__init__`.
2. Write `add_to_order(item)` — appends a `MenuItem` to the cart.
3. Write `get_total()` — loops through `self.items` and returns the sum of all prices as a float.
4. Write `display_order()` — prints each item name, price, and the total at the bottom.
5. Write `clear_order()` — resets `self.items` to an empty list.
6. Test block: create an `Order`, add 2–3 fake `MenuItem` objects, call `display_order()`, then call `clear_order()` and confirm the cart is empty.

Role C — Inventory

1. Create `inventory.py`. Define the `Inventory` class. In `__init__`, set `self.stock` to a dict with at least 6 item names as keys and stock counts as values.
2. Write `check_stock(item)` — returns `True` if `self.stock[item.name] > 0`, otherwise `False`.

3. Write `deduct_stock(item)` — reduces the count for `item.name` by 1.
4. Write `restock(item_name, amount)` — adds `amount` to the stock for `item_name`.
5. Write `stock_report()` — prints every item name and its current quantity in a neat format.
6. Test block: create an `Inventory`, call `check_stock` on an in-stock item, deduct it, then call `stock_report()`.

End-of-Week1 Showcase (3 minutes per team)

Each team presents their class to the class. Your walkthrough must cover:

- Run your test block live — show it working in the terminal
- Explain one design decision you made (e.g. why you structured `__init__` the way you did)
- Show what happens with bad input — what if someone orders an item with no stock? What if the cart is empty?

After all teams present, share your `.py` file(s) with the class (copy file or share in github). Everyone should have access to every team's code before Week 2.

Week 1 deliverable — submit by end of week

- Your completed `.py` file with the class and test block
- A half-page written explanation: what does your class do, and what would break if someone used it wrong?

Week 2 — Select, Integrate & Build the App

Now your team becomes the full development team. You will choose the best `MenuItem`, `Order`, and `Inventory` class from all submissions — they do not have to be your own — and build the working app around them.

Step 1 — Select your classes

Review every team's code from the shared folder. For each of the three classes, pick the version you want to use. You may mix and match freely — you can use Team A's `MenuItem`, Team C's `Order`, and your own `Inventory` if you like.

Selection rationale (written, submitted with Week 2):

- Which team's `MenuItem` did you choose and why?
- Which team's `Order` did you choose and why?
- Which team's `Inventory` did you choose and why?

2–3 sentences per class is enough. You are being graded on whether you actually read and compared the code.

Step 2 — Assign Role D (App Controller)

One person in the team takes on `main.py`. This is the hardest role because it must understand all three classes.

Role D — App Controller (`main.py`)

1. Import your chosen classes: `from menu import MenuItem, MENU, display_menu, get_item, etc.`

2. Create the main loop — a `while True` that shows these options:

```
[1] View menu      [2] Order item    [3] View cart      [4] Checkout      [q] Quit
```

3. Option 1 calls `display_menu()`.

4. Option 2 asks for an item name, calls `get_item()` to find it, calls `check_stock()` to confirm availability, then calls `add_to_order()` and `deduct_stock()`.

5. Option 3 calls `display_order()`.

6. Option 4 (checkout): if the cart is empty, print a message and return to the menu. Otherwise print an itemised receipt, the total, simulate payment accepted, call `clear_order()`, and save the total to a session history list.

7. Option q: before quitting, print a session summary — number of orders completed and total revenue earned.

Integration tip:

You will almost certainly hit a bug where a method returns the wrong type or has a slightly different name than you expected. This is normal. It is also the most important thing you will learn this week — reading someone else's code carefully and adapting to it is a core real-world skill.

Week 2 deliverable — submit by end of week

- `main.py` wired to your chosen classes — app runs end-to-end
- The three chosen `.py` class files (even if unmodified from Week 1)
- Written selection rationale — 2–3 sentences per class explaining your choice
- Week 2 check-in demo — run the app live and place at least one full order through to checkout

Week 3 — Polish, Edge Cases & Final Demo

Your app runs. Now make it solid. This week is about handling the things that go wrong, adding one feature of your choice, and preparing your final presentation.

Required fixes — all teams

- Empty cart checkout: if the user picks Checkout with nothing in the cart, print a friendly message instead of crashing or showing \$0.00
- Item not found: if the user types an item name that is not on the menu, print an error and re-prompt — do not crash
- Out of stock: if `check_stock()` returns `False`, tell the user the item is unavailable and let them choose something else
- Invalid input: if the user types something other than 1/2/3/4/q at the main menu, handle it gracefully

Bonus feature — choose one

Each team picks one bonus feature to add. This is open-ended — design it yourselves.

Option	What it does	Hint
Daily special	One random item is 20% off each session	<i>Use <code>random.choice()</code> on the MENU list</i>
Loyalty points	Every \$5 spent earns 1 point. Print point total on receipt.	<i>Add a points counter to <code>main.py</code></i>
Staff restock mode	Type 'staff' at the main menu to enter a restock screen	<i>Add a branch to the main loop</i>
Order history	Print a full list of every order placed this session before quitting	<i>Append each order dict to a list</i>

Final demo — in class

Each team runs their app live. Each person will present, so divide the demo and presentation. The demo must show:

- Viewing the menu
- Placing orders with at least 2 items
- Attempting to order an out-of-stock item — show the error handling
- Checking out and printing the receipt
- Quitting and showing the session summary
- A brief explanation of which teams' classes you used and one thing you had to fix or adapt

Week 3 deliverable — final submission

- All .py files — clean, commented, working
- A short reflection (half page per person): what did you build, what broke, what would you do differently?
- Live demo in class